Transformers: 2017 vs 2026
(Multimodal LLMs, MoE, Attention at Scale)

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 11, 2026

Table of Contents

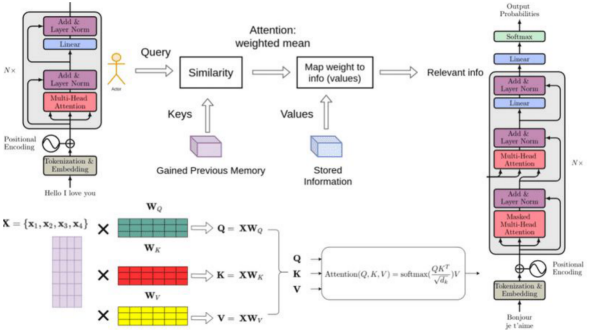
Table of Contents (cont.)

Five Key Engineering Changes (2017 → 2026)

1. **New defaults inside the block:** relative positions (RoPE), normalization moved before each sublayer and simplified to RMSNorm, gated feed-forward blocks (SwiGLU), and biases dropped.
2. **Bandwidth matters more than just speed:** New attention methods like FlashAttention focus on moving data efficiently and using memory well.
3. **From fixed to flexible compute:** Models now adjust their computation and memory use based on the input (using tools like sparse MoE).
4. **From separate to unified input streams:** Instead of having different pieces for text, images, etc., everything is processed together in one sequence, with new practical challenges.
5. **More thinking during use:** Some new models use extra internal steps ("thinking tokens") for harder questions, shifting cost to longer decoding instead of just bigger models.

Plus one thread that is not a change to the transformer: state-space models replace attention with a fixed-size recurrent state. S4 (2021) → Mamba (2023) → Mamba-2, Jamba (2024) → hybrid stacks (2025+).

The 2017 Transformer



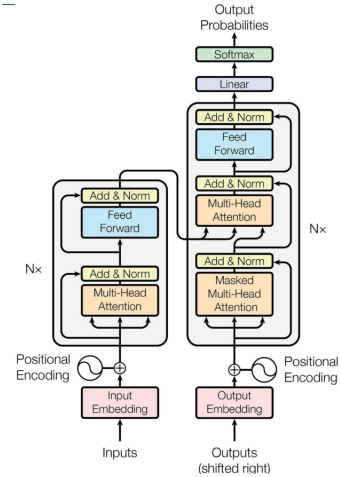
Composite overview slide, source unidentified; the encoder–decoder stacks are after Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

What the 2017 Design Optimized For

- The 2017 target was *training* throughput. Replacing recurrence with self-attention made every position computable at once, which cut the wall-clock time to convergence.
- Nothing in that design was tuned for serving. Attention was written as three passes over an $N \times N$ score matrix in main memory, every head kept its own keys and values, and each generated token cost exactly one forward pass.
- By 2026 the binding constraint has moved: a model is trained once and served billions of times, so memory traffic and cache size decide the cost per token.
- Most of the changes in this lecture follow from that shift, and they leave the mathematics of attention largely intact.

The 2017 Building Blocks (Recap)

- Tokenization and input embeddings
- Sinusoidal position encodings
- Self-attention over Query, Key, Value
- Multi-head attention
- Feed-forward blocks with Add & Norm
- Encoder stack, decoder stack, masked and cross-attention



Source: Vaswani et al., "Attention Is All You Need," NeurIPS 2017, Fig. 1.

Scaled Dot-Product and Multi-Head Attention (Recap)

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

- Every token attends to every other token: quadratic cost in sequence length.
- In 2017 every head carries its own full K and V projections; MQA and GQA later relax this.

2017: Sinusoidal Positional Encoding (Recap)

- Self-attention is permutation-invariant: without extra signal, the model cannot tell token order.
- The 2017 Transformer adds a fixed sinusoidal vector to each input embedding.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

- The encoding is absolute: each position gets its own vector, added once at the input.
- Two problems surfaced at scale: absolute positions generalize poorly beyond the training length, and the signal is injected only once at the input, so every deeper layer receives it indirectly.

2026: From Absolute Positions to RoPE and ALiBi

- **Learnable position embeddings** (BERT, GPT-2): replace the fixed sinusoids with trained vectors. Adapt to the task, but still absolute and still capped at the training length.
- **Rotary Position Embeddings (RoPE)** (Su et al., 2021): instead of adding a position vector at the input, rotate each query and key by a position-dependent angle inside every attention layer.
  - The dot product between a query at position m and a key at position n then depends only on the offset $m - n$: relative position, injected at every layer.
  - Rotary frequencies can be rescaled after training to stretch the context window: position interpolation compresses the position indices (Chen et al., 2023), and YaRN rescales the frequency bands unevenly (Peng et al., 2023).
  - RoPE is the dominant choice in 2026: LLaMA, Mistral, and Qwen all use it.
- **ALiBi** (Press et al., 2022): no position embedding at all. Add a linear penalty to attention scores proportional to token distance, so attention decays with distance by construction. Extrapolates to longer sequences than it was trained on; used by BLOOM and MPT.

The Transformer Block and LayerNorm Placement (Recap)

The 2017 block:

- Multi-head attention
- Add & Norm
- Feed-forward layer
- Add & Norm

Post-LN in the original paper: normalization applied after each sublayer.

Feature	Pre-LN	Post-LN
Stability	✓ More stable	✗ Less stable
Gradient flow	✓ Better	✗ Can explode/vanish
Used in	GPT-3, T5	Original Transformer

By 2020 nearly every large model had moved the norm before the sublayer (Pre-LN): gradients reach early layers directly through the residual path, so deep stacks train without warm-up tricks.

2026: From LayerNorm to RMSNorm

LayerNorm (Ba et al., 2016; used in the 2017 Transformer)**:** center, then scale.

$$\mathrm{LN}(x) = \frac{x - \mu}{\sigma} \odot \gamma + \beta$$

RMSNorm (Zhang & Sennrich, 2019)**:** drop the mean subtraction and the bias; scale by the root mean square alone.

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}(x)} \odot \gamma, \qquad \mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

2026: From LayerNorm to RMSNorm (cont.)

- One reduction over the hidden dimension instead of two, and no bias parameters: cheaper at every layer of every token.
- Quality matches or slightly beats LayerNorm at scale, so centering adds cost without benefit.
- LLaMA, Mistral, and Qwen all combine Pre-LN placement with RMSNorm, the default in 2026.
- Newer stacks also normalize inside attention: Qwen3, Gemma 3, and OLMo 2 apply RMSNorm to queries and keys (QK-Norm) to keep attention logits stable.

2017: The ReLU Feed-Forward Block (Recap)

- Each Transformer block ends with a position-wise feed-forward network: two linear maps with a ReLU between them.

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\, W_2 + b_2$$

- The hidden width is typically $4 \times d_{\text{model}}$, so these blocks hold the largest single share of a Transformer's parameters (about 40% of the 65M base model, and roughly two thirds of the non-embedding parameters in modern decoder-only stacks).
- Attention mixes information across tokens; the feed-forward network transforms each token on its own, with the same weights at every position.

2026: From ReLU to SwiGLU

GLU variants (Shazeer, 2020)**:** replace the single hidden activation with the product of a value path and a gate path, using the gated linear unit of Dauphin et al. (2016) inside the feed-forward block.

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \left(\mathrm{SiLU}(xW_1) \odot xW_3\right) W_2, \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

- Three weight matrices instead of two; the hidden width shrinks to roughly $\frac{2}{3}$ of $4 \times d_{\text{model}}$ so the parameter count stays matched.
- Small but consistent quality gains at equal parameters and compute. The paper offers no theory for why: "we attribute their success, as all else, to divine benevolence."
- Bias terms are dropped at the same time: modern stacks remove b from attention and feed-forward projections with no measurable quality loss.
- LLaMA, Mistral, Qwen, and PaLM all use SwiGLU; ReLU feed-forward blocks have essentially disappeared from frontier models.

Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202.

KV Cache Memory

- While generating (one token per forward pass), each new token needs the key and value tensors of all earlier positions; caching them avoids recomputation, one cache per layer.
- Serving memory consists of model weights plus the KV cache, and under batching every request needs its own cache, often the limiting factor before weights.
- Cost grows linearly with batch size and context length, capping throughput and long-context use; MQA and GQA target this directly.

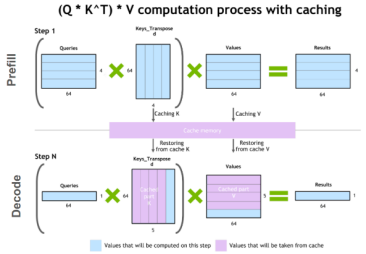
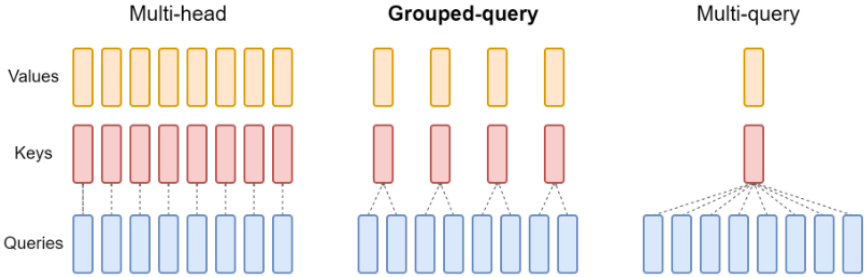


Illustration of the key-value caching mechanism. Verma & Vaidya, NVIDIA Technical Blog (2023).

MHA, MQA, GQA: shrinking the KV cache

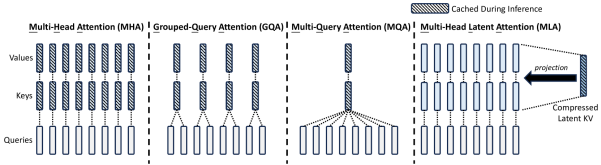


- **Multi-head attention (MHA)**: H separate query, key, and value heads.
- **Multi-query attention (MQA)**: single shared key and value heads for all query heads.
- **Grouped-query attention (GQA)**: shares key and value heads for each *group* of query heads.
- GQA conceptually interpolates between multi-head and multi-query attention.

Reference: Ainslie et al., "GQA," arXiv:2305.13245, Figure 2.

Multi-Head Latent Attention (MLA)

- MQA and GQA shrink the cache by sharing key and value heads; aggressive sharing costs quality.
- MLA (DeepSeek-V2, 2024) instead compresses keys and values into one low-rank latent per token, caches the latent rather than per-head keys and values, and reconstructs them at attention time.
- Expressivity stays close to full MHA with a far smaller cache: DeepSeek-V2 reports 93.3% less KV memory than DeepSeek 67B, already a GQA model. Used across the DeepSeek family (V2, V3, R1).

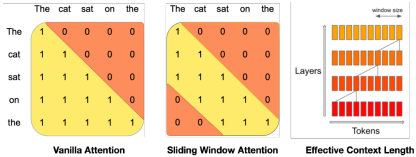


DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv:2405.04434,

Figure 3.

Sliding-Window and Hybrid Attention

- Sliding-window attention (Mistral 7B, 2023): each token attends only to the previous W positions ($W = 4096$ in Mistral), capping the per-layer KV cache at W entries no matter how long the context grows.
- Information still travels further than W: after L layers the receptive field reaches $L \times W$ tokens (32 layers $\times$ 4096 $\approx$ 131k in Mistral 7B).
- The 2026 pattern is hybrid: stacks alternate windowed layers with occasional full-attention layers, keeping long-range retrieval while most layers stay cheap. Gemma 2 alternates 1:1, Gemma 3 uses five local layers per global one, and GPT-OSS alternates banded and dense blocks.



Jiang et al., "Mistral 7B," arXiv:2310.06825, Figure 1.

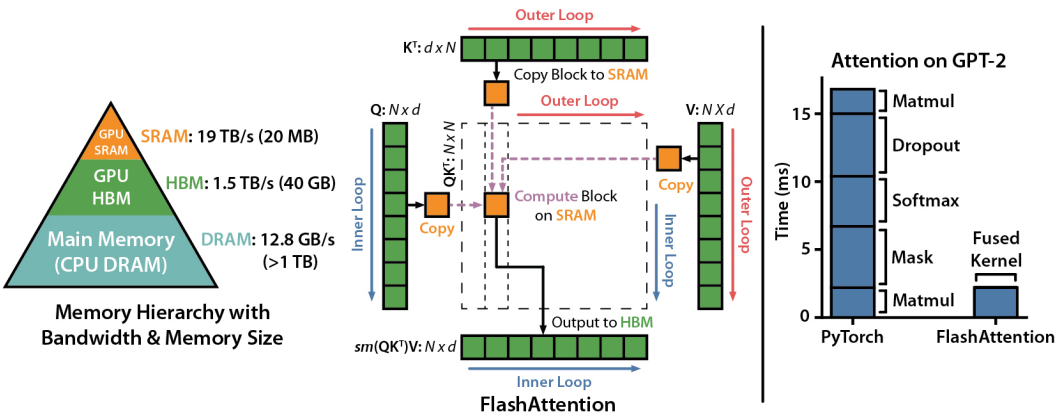
Decoder-only LLMs (post-2017 trend)

- As covered earlier, GPT-style stacks keep only **causal (masked) self-attention**, so one stack replaces the separate encoder and decoder.
- That is the default for language and for most vision–language work: one stream of tokens, one set of weights, and a KV cache that grows in one place.
- Encoder–decoder Transformers survive where the whole input is available before generation starts: machine translation, speech recognition (Whisper), and some captioning models.
- The trade is explicit. Cross-attention reads a fixed encoder output, so its keys and values are computed once per input instead of growing with every generated token.

FlashAttention: The Core Idea

- Standard attention writes the full $N \times N$ score matrix to GPU main memory (HBM), reads it back for the softmax, then reads it again to multiply with V: most of the time goes to memory traffic, not math.
- On-chip SRAM is roughly an order of magnitude faster than HBM but only megabytes in size (on an A100, about 19 TB/s across $\sim$20 MB of SRAM against 1.5–2.0 TB/s across 40–80 GB of HBM).
- FlashAttention (Dao et al., 2022) splits Q, K, V into tiles that fit in SRAM and computes attention block by block with a running softmax, never materializing the $N \times N$ matrix.
- The output is exact (no approximation), memory grows linearly instead of quadratically with sequence length, and wall-clock time drops because IO shrinks.

FlashAttention: Memory Hierarchy and Tiling



GPU memory hierarchy, tiling scheme, and GPT-2 kernel timings. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," arXiv:2205.14135; figure from the official flash-attention repository.

FlashAttention-2: Overview

- **Paper:** Tri Dao (2023), arXiv:2307.08691
- **Highlights:**
  - 50–73% of theoretical peak FLOPs/s on A100, against 25–40% for FlashAttention-1
  - Around 2× faster than FlashAttention-1, itself 2–4× faster than optimized standard attention
- **Techniques:**
  - Fewer non-matmul FLOPs
  - Parallelism over sequence length, not only batch and heads
  - Work partitioning across warps to cut shared-memory traffic
- **Adoption:**
  - Widely used in open training and serving stacks (Mistral among them)
  - The payoff is context length: 16k-context training at the price of 8k

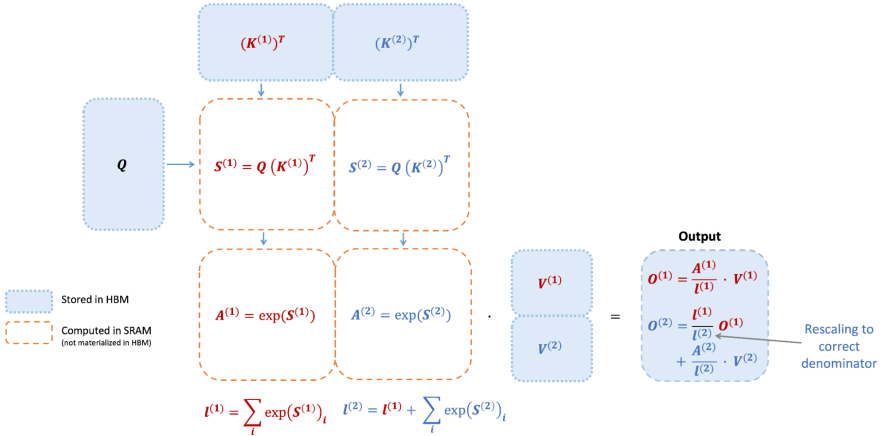
FlashAttention-2: Key Improvements

- **Fewer non-matmul FLOPs:**
  - Algorithm tweaks reduce non-matmul operations (e.g., rescaling, masking).
  - Maximizes use of specialized GPU units (Tensor Cores).
  - Non-matmul FLOPs are up to 16× more expensive than matmul FLOPs.
- **Better Parallelism:**
  - Parallelizes over sequence length in addition to batch size and heads.
  - Improves GPU utilization for long sequences and small batches.

FlashAttention-2: Key Improvements (cont.)

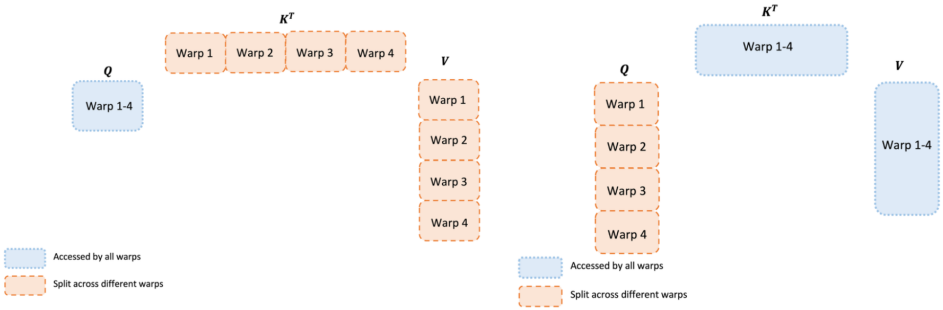
- **Improved Work Partitioning:**
  - Splits Q across warps (instead of K/V), reducing shared memory communication.
  - Yields significant speedup by minimizing synchronization.
- **New Features:**
  - Supports head dimensions up to 256 (FlashAttention-1 stopped at 128).
  - Supports multi-query and grouped-query attention, so kernel speed and a smaller cache combine.

FlashAttention-2: Tiling and Work Partitioning



Tiled forward pass: attention is computed against one K/V block at a time and the output rescaled to the correct denominator, so the $N \times N$ matrix never reaches HBM. Dao, "FlashAttention-2," arXiv:2307.08691, Figure 1.

FlashAttention-2: Tiling and Work Partitioning (cont.)



Work partitioning between warps in the forward pass. FlashAttention-1 (a) splits K and V across warps and must combine partial results through shared memory; FlashAttention-2 (b) splits Q instead, so no warp-to-warp communication is needed. Dao, "FlashAttention-2," arXiv:2307.08691, Figure 3.

FlashAttention-2: Performance Benchmarks

- **Speed:**
  - Up to 2× faster than FlashAttention-1, 9× faster than standard PyTorch attention.
  - Up to 230 TFLOPs/s on A100 and 335 TFLOPs/s on H100.
- **End-to-End Training:**
  - 1.3× speedup over an already FlashAttention-optimized model.
  - Up to 225 TFLOPs/s per A100 (72% model FLOPs utilization).
- **Example Results** (training speed, TFLOPs/s per GPU on 8×A100)**:**

Model	Baseline	FlashAttention	FlashAttention-2
GPT3-1.3B 2k	142	189	196
GPT3-1.3B 8k	72	170	220
GPT3-2.7B 2k	149	189	205
GPT3-2.7B 8k	80	175	225

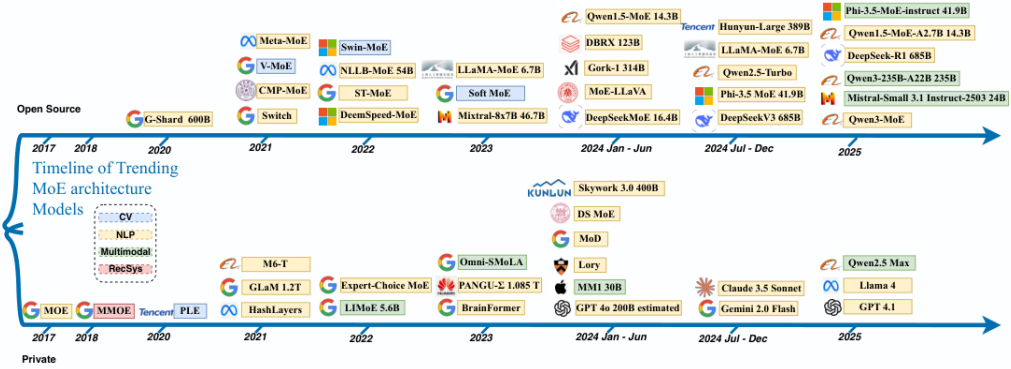
Baseline: Megatron-LM without FlashAttention. Dao, arXiv:2307.08691, Table 1.

Mixture of Experts (Recap)

- As covered earlier, an MoE block replaces the single dense feed-forward MLP with several smaller expert MLPs and a router that sends each token to a few of them.
- What matters for the 2017-to-2026 comparison is the consequence: total parameters and per-token FLOPs stop being the same number. Capacity can grow while the arithmetic done for each token stays bounded.
- This is the flexible-compute item from the roadmap. Sliding windows and MLA make the memory a token costs depend on the architecture; MoE makes the arithmetic it costs depend on the router.

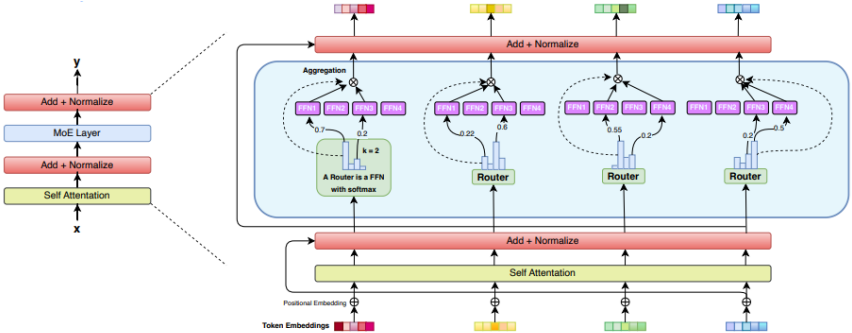
Sources: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181; Kranen & Nguyen, NVIDIA Technical Blog (Mar. 2024).

MoE milestones



Reference: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181, Figure 1.

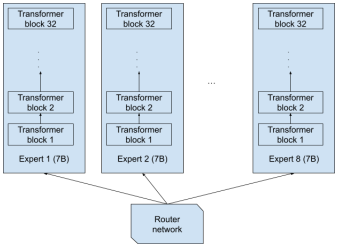
MoE for a decoder-only Transformer



Top-k routing with $k = 2$ over four expert FFNs, in place of the dense feed-forward sub-block. *Reference:* Zhang et al., arXiv:2507.11181, Figure 3.

Example: Mixtral 8×7B, a common misconception

- The name invites the arithmetic $8 \times 7\text{B} = 56\text{B}$, and the reading "eight separate 7B models." The released architecture is neither.
- Attention layers, norms, and embeddings are shared; only the feed-forward sub-block is replicated eight times. Mixtral 8×7B is 46.7B parameters in total and spends 12.9B on each token, because the router fires only two experts per token.



The mental model the name suggests, which is *not* the released design. Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article. Parameter counts: Jiang et al., "Mixtral of Experts," arXiv:2401.04088.

MoE: Most Preferred Tokens By Experts

Word clouds of the tokens each Mixtral expert processes most often, at layers 1, 16, and 32. The routing keys on token identity rather than on topic or domain, so "experts" are not subject specialists. *Reference:* Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article.

Attention's cost, and the recurrent alternative

- Everything in this deck so far keeps attention and makes it cheaper. This section covers the one thread that asks a different question: does the sequence mixer have to be attention at all?
- The motivation is the KV cache from earlier. Attention keeps every past token individually addressable, which is exactly why it can retrieve anything, and exactly why serving memory grows with context and batch size.
- A recurrence trades the other way: history is compressed into a fixed-size state h_t, so a decoding step costs the same no matter how much context precedes it. 2017-era RNNs had that property, but they trained sequentially and lost long-range detail.
- The state-space line asks for both at once: a constant-size state *and* training that is parallel across the sequence. (Shrinking or paging the cache is the other answer to the same problem, and is covered later in the course.)

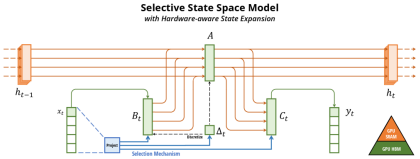
S4: a structured linear recurrence

- Start from a continuous linear system $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$ and discretize it with a step size Δ: $h_t = \bar{A}h_{t-1} + \bar{B}x_t$, $y_t = Ch_t$. A linear RNN, with no nonlinearity in the state update.
- Because the parameters are the same at every position (linear time-invariant), unrolling gives a convolution $y = K * x$ with $K = (C\bar{B},\ C\bar{A}\bar{B},\ C\bar{A}^2\bar{B},\ \ldots)$: training is one global convolution instead of L sequential steps.
- S4 makes that practical. Initialize A from HiPPO theory so the state holds a compressed summary of the history, and parameterize it as diagonal-plus-low-rank so K can be computed stably as a Cauchy kernel.
- It won on the regime transformers struggled with: state of the art on every Long Range Arena task, including the first solution of Path-X at length 16k.
- The limitation is inside "time-invariant": one fixed kernel applies to every token, so the model cannot keep or discard something because of what that token is.

Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," arXiv:2111.00396v3.

Mamba: selection breaks time-invariance

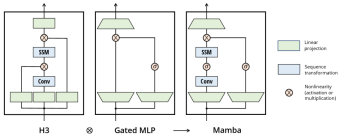
- Mamba's one change: let B, C, and the discretization step Δ be functions of the current token. A stays fixed, but Δ_t modulates it through the discretization, so the effective $\bar{A}_t$ and $\bar{B}_t$ now vary along the sequence.
- Read Δ_t as a gate. Large means reset the state toward the current token; small means ignore this token and hold what the state already has: content-based filtering inside a linear recurrence.
- The evidence is synthetic and sharp: time-invariant models solve copying at fixed spacing, but fail selective copying and induction heads, which require deciding what to keep from the content.



Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752v2, Figure 1.

The Mamba block and the parallel scan

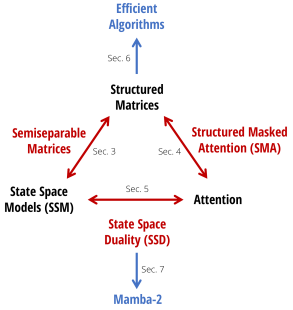
- Selectivity costs the convolution: per-token parameters leave no single kernel, so the recurrence must be run. Mamba runs it as a **hardware-aware parallel scan** keeping the expanded state in SRAM and recomputing it in the backward pass instead of writing it to HBM, FlashAttention's argument applied to a recurrence.
- Cost model: training is linear in sequence length and still parallel; at inference the state is fixed size, so each step is $O(1)$ memory and L tokens cost $O(L)$ time, with no cache that grows.
- Reported payoff: roughly $5\times$ the generation throughput of a same-size Transformer, with Mamba-3B matching Transformers about twice its size.



The block folds the H3 SSM and a gated MLP into one unit. Gu & Dao, arXiv:2312.00752v2, Figure 3.

Mamba-2: state-space duality

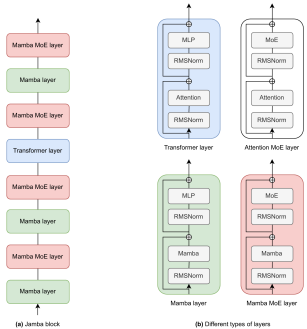
- Write a linear SSM's whole sequence map as one matrix, $y = Mx$. That M is **semiseparable**: its off-diagonal blocks are low rank.
- Multiplying by M as a recurrence is linear in L; materializing it and applying it row by row is a masked-attention form, quadratic in L. Structured state-space duality says these are two algorithms for one computation, not two model families.
- Mamba-2 cashes that in: restricting A to a scalar times the identity lets the layer run as large matmuls on tensor cores, 2–8× faster than Mamba's selective scan and with a far larger state.
- So the efficient-attention thread and the SSM thread met in the middle. Linear attention, gated recurrences, and selective SSMs are one design space.



Dao & Gu, "Transformers are SSMs," arXiv:2405.21060v1, Figure 1.

Hybrids: Jamba, Griffin, and after

- **Jamba** (2024) interleaves Mamba layers, Transformer layers, and MoE. Its blocks are $l = 8$ layers at a 1:7 attention-to-Mamba ratio, with MoE every second layer: 52B total, 12B active, 256K context.
- The number that explains the design: at 256K tokens the KV cache is 4 GB, against 32 GB for Mixtral, because only the attention layers keep one.
- **Griffin** (2024) swaps the SSM for a real-gated linear recurrent unit interleaved with local sliding-window attention. Hawk (pure recurrence) exceeds Mamba's reported results; Griffin matches Llama-2 on 6× fewer tokens.
- By 2025 hybrids are the shipping form, the sliding-window recipe with a cheaper layer type: Zamba2, Nemotron-H, Falcon-H1, RecurrentGemma.

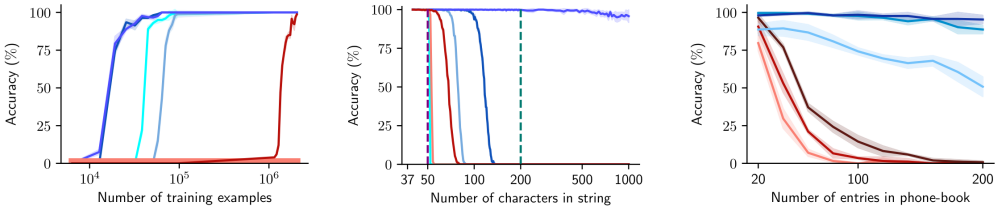


Lieber et al., "Jamba: A Hybrid Transformer-Mamba Language Model," arXiv:2403.19887v1, Figure 1.

Scorecard: what a fixed state costs

- A fixed-size state is a summary, and whatever the summary dropped is gone. Attention keeps every token addressable, and pays memory for that exact recall.
- "Repeat After Me" makes it concrete: any fixed-state model provably fails to copy strings carrying more bits than its state holds, and empirically Transformers learn copying from far fewer samples, generalize to longer strings, and beat same-size Mamba at in-context lookup even where Mamba has the lower perplexity.
- Jamba's own ablation agrees. At 1.3B, pure Mamba scores 48.8 on IMDB against 84.1 for pure attention, often failing to produce the answer format at all; the hybrid recovers it at 90.9. The authors read this as weak in-context learning, tied to the absence of induction heads.
- The wins are real but narrower than the headlines: throughput and memory at long context, streaming inference, and modalities such as audio and genomics.

Copying and lookup: Transformers vs. SSMs



(a) Copying: training efficiency. (b) Copying: length generalization (c) Lookup with pretrained models

Transformers (blues) against fixed-state models (reds) on copying and retrieval. (a) sample efficiency when learning to copy; (b) accuracy on strings longer than those seen in training, with the training length and context window marked; (c) pretrained Pythia against pretrained Mamba at comparable sizes, retrieving a number from an in-context phone book. *Reference:* Jelassi et al., "Repeat After Me: Transformers are Better than State Space Models at Copying," arXiv:2402.01032v2, Figure 1.

Verdict (2026)

- Attention was not replaced. What survived from this thread is a layer type, not an architecture: SSM and gated-recurrence layers carry most of the depth, and a handful of attention layers keep exact retrieval available.
- That is the same economics as the sliding-window hybrids earlier in this deck. Make the common layer cheap in memory traffic, then pay for a few expensive layers where the capability actually lives.
- Structured state-space duality dissolved the border between the two research programs, so "attention versus SSM" now reads better as a choice of mixing matrix inside one family.
- Which is this deck's recurring lesson once more: the changes that stuck are the ones that move less memory per token. Byte-level and diffusion language models change the tokenizer and decoding order rather than the mixer; both come later in the course.

Vision–Language Models: naming and motivation (Recap)

- As covered earlier, pairing vision with language lets a user specify a new task in words ("what species of bird is this?", "read the sign in the corner") instead of collecting a fresh label set for every concept.
- Nomenclature (informal): an **LLM** is language-first; a **VLM** adds a vision encoder so the stack can condition on images; an **MLLM** usually means several modalities (image, video, audio) in one family.
- The architectural question this section answers is narrower: at what point in the stack do image tokens meet the language model, and what does each answer cost?

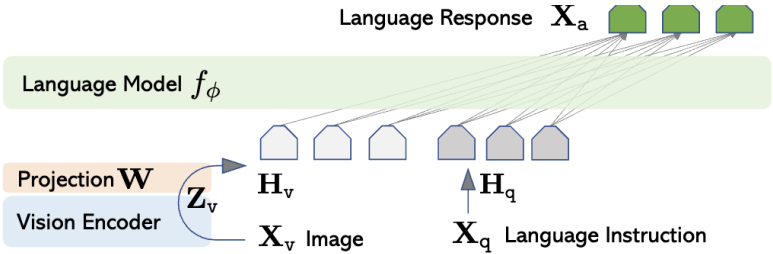
Bandaru, *Vision Language Models* (blog, Aug. 2025), rohitbandaru.github.io/blog/Vision-Language-Models/.

Fusion Patterns: Where do image tokens meet the LLM?

- Early fusion: build one long token sequence (visual tokens + text tokens) and run attention over the mixture. This is what most "native" VLMs aim for today.
- Middle fusion: insert vision cross-attention inside some transformer blocks. This is the usual choice when retrofitting vision onto a stack that was already trained on text alone.
- Late fusion: encode each modality separately and compare the two embeddings at the end. Right for retrieval and zero-shot scoring, as in CLIP, but too weak for open-ended captioning or VQA, because the language model never receives rich visual state.
- Images create many tokens; good designs compress visual length (pooling, grouping, tiling) before the LLM sees them.

Early fusion: linear or MLP projector

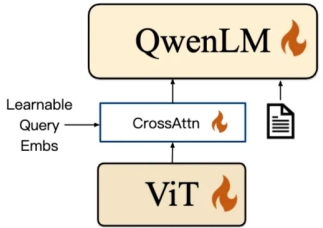
- Freeze (or lightly train) a strong ViT; map its patch tokens through a small projector into the LLM embedding width, then concatenate with text tokens.
- Simple, stable, and very common in open recipes. The projector is a single linear map in LLaVA v1 and PaliGemma 2, and a two-layer MLP in LLaVA-1.5, DeepSeek-VL, and LLaMA 4.



Liu et al., LLaVA, arXiv:2304.08485, Figure 1; via Bandaru, *Vision Language Models* (blog).

Early fusion: cross-attention adapter

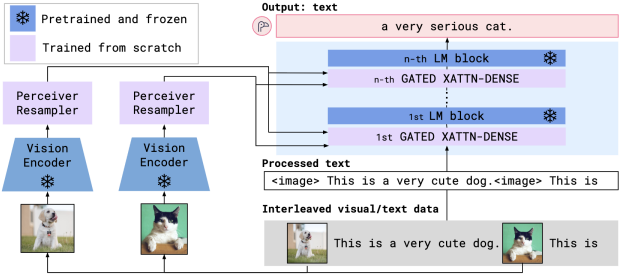
- A small adapter uses learned query vectors that cross-attend to the ViT tokens and emit a fixed, short set of visual summaries (256 in Qwen-VL) that are then concatenated with the text tokens.
- The cross-attention lives in the adapter, outside the language model, so the LLM stack is unchanged and still sees one mixed sequence. That is what keeps this early rather than middle fusion.



Bai et al., Qwen-VL, arXiv:2308.12966, Figure 3 (multi-task pre-training stage); via Bandaru, *Vision Language Models* (blog).

Middle fusion: Flamingo

- Keep the pretrained vision encoder and LLM frozen; insert Perceiver resamplers (compress a variable number of vision features to a fixed 64 tokens) and gated cross-attention blocks inside the LLM stack.
- The vision encoder here is an NFNet-F6, not a ViT: Flamingo predates the convention of pairing every LLM with a vision transformer.



Alayrac et al., Flamingo, arXiv:2204.14198, Figure 3; via Bandaru, *Vision Language Models* (blog).

Middle fusion: LLaMA 3

- **ViT encoder:** ViT-H/14 (630M parameters, 32 blocks) trained CLIP-style for image–text alignment; features are taken from several intermediate layers, not only the last.
- **Decoder:** a cross-attention layer after every fourth self-attention layer, tanh-gated with the gates initialized to zero so the vision path starts as a no-op and is learned in. The language weights are never updated.
- **Why middle fusion:** it suits LLaMA 3's text-first design, since vision is added to a finished language model. LLaMA 4 moves to early fusion so that text and image tokens can be trained together from the start.

Grattafiori et al., "The Llama 3 Herd of Models," arXiv:2407.21783, Sec. 7.

Reasoning Models and Thinking Tokens (Recap)

- A 2017 Transformer spends one forward pass per generated token: every query receives the same compute, easy or hard.
- Reasoning models (OpenAI o1, DeepSeek-R1) generate a long chain of thought before the final answer, so hard questions receive more decode steps than easy ones.
- Accuracy scales with test-time compute: thinking longer or sampling more candidate solutions buys accuracy without retraining.
- R1 shows this allocation can be learned: reinforcement learning on verifiable rewards teaches the model when and how long to think.

DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv:2501.12948.

What Thinking Tokens Mean for the Architecture

- **Cost model:** serving cost is now dominated by decode length, not just parameter count. A smaller model that thinks longer can beat a larger one that answers immediately.
- **KV cache:** every thinking token adds cache entries, so long traces multiply the memory pressure that GQA, MLA, and sliding-window attention exist to relieve.
- **Kernels:** attention runs over reasoning traces tens of thousands of tokens long, exactly the regime FlashAttention targets.
- **Sparsity:** MoE keeps per-token FLOPs bounded, so capacity can grow without making each thinking token slower.
- The 2026 design trade: spend compute at training time (a bigger model) or at inference time (longer thinking), tuned per deployment.

Summary: Transformers 2017 vs. 2026

Component	2017 Transformer	2026 consensus
Attention mechanism	Multi-Head Attention (MHA)	Grouped-Query Attention (GQA) / Multi-Head Latent Attention (MLA)
Position encoding	Absolute sinusoidal (or learned absolute)	Rotary Positional Embeddings (RoPE)
Normalization	Post-Layer Normalization (post-LN)	Pre-Layer Normalization with RMSNorm
Activation	ReLU in the FFN	SwiGLU / gated linear units
Bias terms	Biases in linear layers common	Bias-free or minimal-bias architectures
Parameter density	Dense (all parameters active per forward)	Sparse activation (e.g., Mixture-of-Experts)
Attention span	Every token attends to all positions	Hybrid stacks: sliding-window layers + occasional full-attention layers
Inference compute	Fixed: one forward pass per token	Test-time scaling: thinking tokens on hard queries

Summary

- The slowest part is often getting data in and out of memory, not just the math.
- New models (2024–2026) are designed together, optimizing attention, normalization, position encoding, sparsity (MoE), and serving (cache size, kernels).
- The main goals: cut down on moving data, adjust compute to the input's length and type, and add more parameters without needing a lot more computation per token.
- One thread stayed outside the block: state-space models (S4 $\rightarrow$ Mamba $\rightarrow$ Mamba-2) swapped attention for a fixed-size recurrent state, and by 2026 survive mainly as the cheap layer type inside hybrid stacks that keep a few attention layers for exact recall.

References and Further Reading

1. Vaswani et al., “Attention Is All You Need”, NeurIPS 2017, arXiv:1706.03762.
2. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding”, arXiv:2104.09864.
3. Chen et al., “Extending Context Window of Large Language Models via Positional Interpolation”, arXiv:2306.15595; Peng et al., “YaRN: Efficient Context Window Extension of Large Language Models”, arXiv:2309.00071.
4. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models”, arXiv:2302.13971; “LLaMA 2: Open Foundation and Fine-Tuned Chat Models”, arXiv:2307.09288.
5. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need”, arXiv:1911.02150 (MQA).
6. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”, arXiv:2305.13245.
7. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models”, JMLR 2022, arXiv:2101.03961.
8. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, NeurIPS 2022, arXiv:2205.14135.
9. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”, ICLR 2024, arXiv:2307.08691.
10. Liu et al., “Visual Instruction Tuning” (LLaVA), arXiv:2304.08485.

References and Further Reading (cont.)

11. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, arXiv:2309.06180 (vLLM).
12. Zhang et al., “Mixture of Experts in Large Language Models”, arXiv:2507.11181.
13. Bai et al., “Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond”, arXiv:2308.12966.
14. Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning”, NeurIPS 2022, arXiv:2204.14198.
15. Grattafiori et al., “The Llama 3 Herd of Models”, arXiv:2407.21783.
16. Jiang et al., “Mixtral of Experts”, arXiv:2401.04088.
17. Dauphin et al., “Language Modeling with Gated Convolutional Networks”, arXiv:1612.08083 (gated linear units).
18. Shazeer, “GLU Variants Improve Transformer”, arXiv:2002.05202 (SwiGLU).
19. Ba, Kiros & Hinton, “Layer Normalization”, arXiv:1607.06450.
20. Zhang & Sennrich, “Root Mean Square Layer Normalization”, NeurIPS 2019, arXiv:1910.07467.
21. Press et al., “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation”, arXiv:2108.12409 (ALiBi).

References and Further Reading (cont.)

22. DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", arXiv:2405.04434 (MLA).
23. Jiang et al., "Mistral 7B", arXiv:2310.06825 (sliding-window attention).
24. DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", arXiv:2501.12948.
25. Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog, 2024, developer.nvidia.com/blog/....
26. Bandaru, "Vision Language Models," blog, Aug. 2025, rohitbandaru.github.io/blog/Vision-Language-Models/.
27. Sorte (Medium), Transformer architectures in 2026: foundations and resources.
28. Patro & Agneeswaran, "LLMOrbit: A Circular Taxonomy of Large Language Models, From Scaling Walls to Agentic AI Systems", 2026, arXiv:2601.14053.
29. Aman (Medium), Inside frontier open-weight LLM architectures.
30. MDPI survey, Mapping the LLM landscape (architectures and benchmarks).
31. MIT course notes, Transformers (6.390).
32. Shui, MHA vs. MQA vs. GQA.

References and Further Reading (cont.)

33. DataHacker.rs, Modern transformer architectures (LLMs from scratch series).
34. Rishiraj Acharya, Hugging Face blog, What changed in the Transformer architecture.
35. Tan, The crystallization of transformer architectures (2017–2025).
36. Bandaru, Transformer design guide (modern architecture).
37. NVIDIA, Mastering LLM techniques: inference optimization.
38. Genc (Medium), KV cache, PagedAttention, and MLA.
39. PythonAlchemist, Attention variants explained (MHA, GQA, MQA, MLA, SWA, ...).
40. Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision", NeurIPS 2024, arXiv:2407.08608 (PDF).
41. Lightly, Introduction to vision–language models.
42. Stanford CS224N, NLP with deep learning (course hub).
43. Marek Suppa, NLP 2026 (teaching resources).
44. Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces", ICLR 2022, arXiv:2111.00396 (S4).
45. Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces", arXiv:2312.00752.

References and Further Reading (cont.)

46. Dao & Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”, ICML 2024, arXiv:2405.21060 (Mamba-2 / SSD).
47. Lieber et al., “Jamba: A Hybrid Transformer-Mamba Language Model”, arXiv:2403.19887.
48. De et al., “Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models”, arXiv:2402.19427.
49. Jelassi et al., “Repeat After Me: Transformers are Better than State Space Models at Copying”, ICML 2024, arXiv:2402.01032.
50. Glorioso et al., “The Zamba2 Suite: Technical Report”, arXiv:2411.15242; NVIDIA, “Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models”, arXiv:2504.03624; “Falcon-H1”, arXiv:2507.22448; “RecurrentGemma”, arXiv:2404.07839.